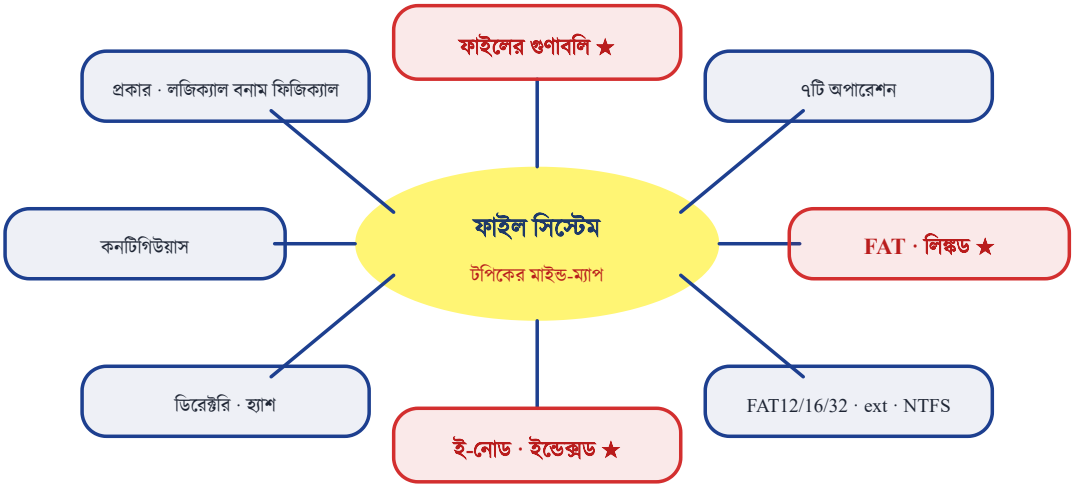
ফাইল সিস্টেম ও কেস স্টাডি

গুণাবলি · FAT · ই-নোড · ডিরেক্টরি · ফ্রি স্পেস · জার্নালিং · UNIX, NT, Linux



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, নীল = সহায়ক বিষয়

সিলেবাসের যে অংশ

অপারেটিং সিস্টেম: "… **ফাইল সিস্টেম: ফাইল, ডিরেক্টরি, নিরাপত্তা, সুরক্ষা**; **অপারেটিং সিস্টেমের কেস স্টাডি**"। প্রসেস, শিডিউলিং, মেমরি, I/O ও ডেডলক এই অধ্যায়ের অন্যান্য লেসনে আছে।

হাই-ইল্ড তালিকা

① ৮টি গুণাবলি; ৭টি অপারেশন
② কনটিগিউয়াস = base + n, বাইরের খণ্ডন
③ FAT = লিঙ্কড, লিঙ্ক থাকে টেবিলে
④ ই-নোড: ১০ ডাইরেক্ট + ১, ২, ৩ ইন্ডারেক্ট
⑤ FAT12 2^{12} · FAT16 2^{16} · FAT32 2^{28}
⑥ জার্নাল = write-ahead log, মেটাডেটা
⑦ NT → রেজিস্ট্রি; Linux → মডিউল

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

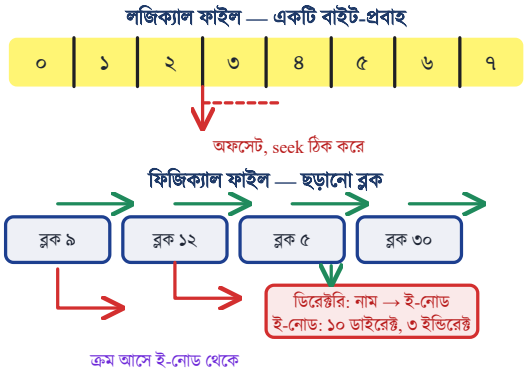
সূচিপত্র

✓ ফাইল ও গুণাবলি … পৃ. ২
✓ ৭টি অপারেশন ও অ্যাক্সেস … পৃ. ৩
✓ কনটিগিউয়াস ও লিঙ্কড (FAT) … পৃ. ৪
✓ ইন্ডেক্সড ও ই-নোড … পৃ. ৫
✓ ডিরেক্টরি গঠন … পৃ. ৬
✓ ট্রেস: open → read → close … পৃ. ৭
✓ ফ্রি স্পেস ও সুরক্ষা … পৃ. ৮
✓ FAT/NTFS/ext/XFS, জার্নাল … পৃ. ৯
✓ কেস স্টাডি: DOS, UNIX, NT, Linux … পৃ. ১০
✓ রাতের পাতা … পৃ. ১১
✓ অনুশীলন MCQ + লিখিত … পৃ. ১২

১ ফাইল সিস্টেম ও ফাইল

সংজ্ঞা: **ফাইল সিস্টেম (file system)** হলো অপারেটিং সিস্টেমের সেই পদ্ধতি যা সেকেন্ডারি স্টোরেজে ডেটা **নাম, সংরক্ষণ, ভাগ ও সুরক্ষা** দেয়, যাতে প্রোগ্রাম কাঁচা ব্লক না দেখে ফাইলের গাছ দেখে। এর দুই অংশ: **ফিজিক্যাল** (কোন ব্লকে কোন ডেটা) ও **লজিক্যাল** (প্রোগ্রাম ফাইলের সঙ্গে কী করতে পারে)। ডিস্ক আগেই **512 B – 4 KiB** ব্লকে ভাগ করা।

গুণাবলি	এতে থাকে	পরীক্ষার ফাঁদ
১ নাম	প্রতীকী নাম	ডিরেক্টরি এন্ট্রিতে, ই-নোডে নয়
২ শনাক্তকারী	OS যে নম্বর ব্যবহার করে	ই-নোড নং · প্রথম ক্লাস্টার
৩ প্রকার	টেক্সট, বাইনারি, এক্সিকিউটেবল	অক্ষর বনাম প্যাকড় রেকর্ড
৪ অবস্থান	ডেটা ব্লকের পয়েন্টার	শুরুর ব্লক, শৃঙ্খল বা ইন্ডেক্স
৫ আকার	বাইট ও ব্লকে দৈর্ঘ্য	নির্দিষ্ট প্রস্থ, তাই আগ্রা লাগে
৬ সুরক্ষা	কে পড়তে বা লিখতে পারে	মোড বিট · ACL · ACE
৭ সময়চিহ্ন	তৈরি, পরিবর্তন, প্রবেশ	UNIX: atime, mtime, ctime
৮ মালিক	ব্যবহারকারী ও গ্রুপের আইডি	শুধু root বদলাতে পারে



প্রোগ্রাম দেখে ০, ১, ২, ৩ …; ডিস্কে আছে ৯, ১২, ৫, ৩০

লজিক্যাল ফাইল = অফসেট ০ থেকে শুরু করে প্রোগ্রাম যে বাইটগুলো দেখে। **ফিজিক্যাল ফাইল** = ডিস্কের প্রকৃত ব্লকগুলো + সেগুলোর দিকে তির করা ডিরেক্টরি এন্ট্রি বা ই-নোড। **লজিক্যাল → ফিজিক্যাল রূপান্তরই ফাইল অর্গানাইজেশন**।

প্রকার: টেক্সট (.txt .md .csv) · **বাইনারি** (.jpg .mp4) · **এক্সিকিউটেবল** (মেশিন কোড; UNIX-এ ELF) · **বিশেষ** (ডিভাইস, FIFO, সকেট: /dev/sda, /dev/null)।

ফাইলের ভেতরে কোথায় "রেকর্ড" শেষ হয় কিছুই লেখা থাকে না — তা ঠিক করে **ফরম্যাট**, তাই একই ফাইল বাইটে, রেকর্ডে বা ব্লকে পড়া যায়।

মনে রাখার কৌশল: "ই-নোডে **নাম ছাড়া সব** আছে।" গুণাবলি ১ ডিরেক্টরিতে, ২–৮ ই-নোডে। এই একটি ভাগাভাগিই হার্ড লিঙ্ক, *ln -s* ও প্রতিটি পাথ অনুসন্ধান বোঝায়।

পরীক্ষার ফাঁদ: ফাইলের **শনাক্তকারী নাম নয়**, আর **সুপারব্লক (superblock)** ফাইলের মেটাডেটা নয় — সে পুরো ভলিউম বর্ণনা করে। ফাইল সিস্টেম **সেকেন্ডারি** স্টোরেজের বিষয়; RAM-এ থাকে বাফার।

সমাধানকৃত উদাহরণ ✍ একটি ডিরেক্টরি লাইন বিশ্লেষণ: *-rwxr-xr-- 1 rahim exam 8200 Sep 9 10:14 report.txt* → **-** সাধারণ ফাইল (d মানে ডিরেক্টরি) · **rwxr-xr-- = 7 5 4**: মালিক rahim rwx, গ্রুপ exam r-x, অন্যরা r-- · লিঙ্ক সংখ্যা 1 · আকার **8 200 B** = 512 B-এর ১৭টি ব্লক।

২ ফাইলের সাতটি প্রমিত অপারেশন

সংজ্ঞা: ফাইল একটি **বিমূর্ত তথ্য-প্রকার (abstract data type)**: নাম দেওয়া, সুরক্ষিত বাইটের ধারা। প্রতিটি অপারেটিং সিস্টেম একে **ঠিক সাতটি** অপারেশন দেয়, এবং প্রতিটি সিস্টেম কল তারই একটি।

অপারেশন	কী করে	সাধারণ কল
তৈরি	নতুন খালি ফাইল বানিয়ে এন্ট্রি দেয়	creat(), open(O_CREAT)
মুছে ফেলা	এন্ট্রি সরায়, ব্লক মুক্ত করে	unlink(), remove()
খোলা	নাম মেলে, মেটাডেটা পড়ে, অধিকার দেখে	open()
বন্ধ করা	বাফার ফালাশ, ডেসক্রিপ্টর ছাড়ে	close()
পড়া	অফসেট থেকে বাইট বাফারে নেয়	read(), fread()
লেখা	বাফারের বাইট অফসেটে লেখে	write(), fwrite()
সিঙ্ক (seek)	অফসেট বদলায়: SET / CUR / END	lseek(), fseek()

বাইট-প্রবাহের মডেল ✍️: সবকিছু লাগে **একটি** সংখ্যার: অফসেট, শুরুতে **0**। read/write তা এগিয়ে নেয়; seek যেকোনো জায়গায় সেট করে।

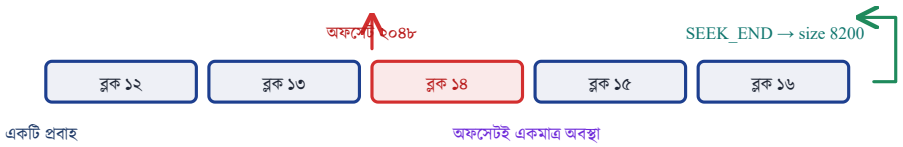
```
fd = open("/home/rahim/report.txt", O_RDONLY);
n = read(fd, buf, 512);       /* 0 → 512 */
p = lseek(fd, 0, SEEK_END);   /* p = size */
close(fd);
```

ধারাবাহিক (sequential) অ্যাক্সেস শুরু থেকে রেকর্ড ধরে রেকর্ড পড়ে — টেপের জন্য এটিই একমাত্র উপায়। **সরাসরি (direct)** অ্যাক্সেস লজিক্যাল ব্লক *n*-কে **O(1)**-তে ফিজিক্যাল ব্লকে বদলে।

অর্গানাইজেশন	ব্লক n মেলে
কনটিগিউয়াস	start + n
লিঙ্কড / FAT	FAT[n]
ইন্ডেক্সড / ই-নোড	ইন্ডেক্স এন্ট্রি n

মনে রাখার কৌশল: "তৈরি করো, তারপর খোলো; খোলার পর পড়ো; শেষে বন্ধ করো।" শুধু read ও write বাইতে ছোঁয় — **seek** কাস্টর সরায়, ডেটা নয়।

পরীক্ষার ফাঁদ: **append** আলাদা অপারেশন নয় — অফসেট শেষে ঠেকিয়ে **write**। **rename** ডিরেক্টরি কাজ। আর **close** তালিকায় আছে।

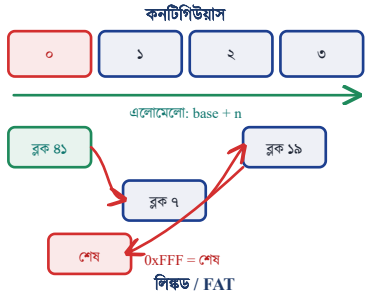


seek ব্লক থেকে ব্লকে কাস্টর সরায়, তারপর read/write আরেকটু সরায়

সমাধানকৃত উদাহরণ ✍️ অফসেট থেকে ডিস্কের ঠিকানা: একটি **কনটিগিউয়াস** ফাইল **ব্লক ১২**-তে শুরু, ব্লক আকার **512 B**। *lseek(fd, 2048, SEEK_SET)* → লজিক্যাল ব্লক 2048 ÷ 512 = **4**, ব্লকের ভেতরে 2048 mod 512 = **0**; তাই পড়া শুরু হয় **ব্লক ১৬** থেকে, 1 KB পড়লে **ব্লক ১৬ ও ১৭** ছোঁয়।

৩ ফাইল অর্গানাইজেশন ১ — কনটিগিউয়াস ও লিঙ্কড

সংজ্ঞা: **ফাইল অর্গানাইজেশন** ঠিক করে কোন ডিস্ক ব্লকগুলো একটি ফাইলের, অর্থাৎ লজিক্যাল ব্লক নম্বর কীভাবে ফিজিক্যাল ব্লকে যায়। এখানেই **অ্যাক্সেসের গতি** ও **ফাঁকা জায়গা ব্যবহারের স্বাধীনতা**-র মধ্যে ভারসাম্য ঠিক হয়।



কনটিগিউয়াস: এক সিক · লিঙ্কড: শৃঙ্খল বেয়ে যাওয়া

কনটিগিউয়াসে ব্লকগুলো **পাশাপাশি** থাকে, তাই একটি সিক ও পরপর পড়াই পুরো ফাইলের চলে। লিঙ্কডে ব্লক ছড়ানো থাকে ও প্রতিটি ব্লকে পরের ঠিকানা থাকে, তাই ফাইল যেকোনো জায়গায় বাড়ে।

বিষয়	কনটিগিউয়াস	লিঙ্কড
ডিরেক্টরিতে থাকে	শুরুর ব্লক + দৈর্ঘ্য	শুধু প্রথম ব্লক
এলোমেলো অ্যাক্সেস	**O(1)**: base + n	O(n): শৃঙ্খল বেয়ে যাওয়া
পরে বড় হলে	সরিয়ে আনতে হয়	যেকোনো জায়গায় স্বাধীন
খণ্ডন	**বাইরের** (external)	বাইরে কোনোটাই নয়
খরচ ও ক্ষতি	পয়েন্টার নেই; একটি ব্লক হারায়	একটি পয়েন্টার; ফাইল ছোঁ হয়

৪ FAT — টেবিলসহ লিঙ্কড অ্যালোকেশন

সংজ্ঞা: **ফাইল অ্যালোকেশন টেবিল (FAT)** হলো এমন লিঙ্কড অ্যালোকেশন যেখানে **ভলিউমের প্রতিটি ব্লকের** "পরের ব্লক" পয়েন্টার ডেটা ব্লকের ভেতরে না রেখে **ভলিউমের শুরুতে একটি টেবিলে** রাখা হয়। এন্ট্রি i-তে থাকে একই ফাইলের ব্লক i-এর পরের ব্লক; **0 = ফাঁকা**, **0xFFF = শৃঙ্খলের শেষ**।

মনে রাখার কৌশল: "একটি FAT হলো **লিঙ্কড** অ্যালোকেশন, যার লিঙ্কগুলো একটি **টেবিলে** সরানো হয়েছে।"

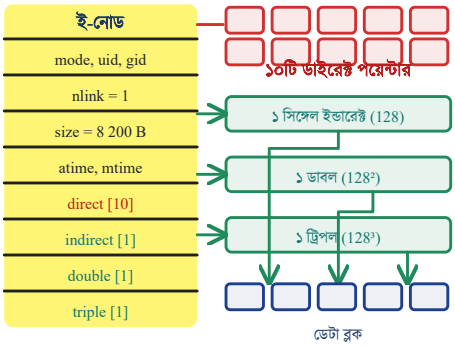
পরীক্ষার ফাঁদ: FAT কোনো কথায় কনটিগিউয়াস বা ইন্ডেক্সড নয়। এটি **প্রতি ফাইলে নয়, প্রতি ভলিউমে** একটি।

সমাধানকৃত উদাহরণ ✍️ FAT-এর খরচ কত?: 1.44 MB ফ্লপিতে 2 880 সেক্টর × 512 B: **1** বুট সেক্টর + **2 × 9** FAT12 সেক্টর + **14** রুট ডিরেক্টরি সেক্টর, তাই ডেটা ক্লাস্টার = 2 880 − 1 − 18 − 14 = **2 847**। যাচাই: 2 847 × 12 বিট = 4 271 B → **9 সেক্টর** ✓। ওভারহেড 33/2 880 ≈ **1.15%**, আর FAT-এর দ্বিতীয় কপি ক্র্যাশে ডিস্ক গড়ে দেয়।

সমাধানকৃত উদাহরণ ✍️ অ্যালোকেশন-সাইজ সমস্যা: **4 096 B** ফাইলে 512-B ব্লকে ৮টি ব্লক লাগে, তাই কনটিগিউয়াস অ্যালোকেশনকে তৈরির সময়ই **দৈর্ঘ্য ঘোষণা** করতে হয়। 16-বিট দৈর্ঘ্য ফিল্ডে সর্বোচ্চ 65 535 ব্লক = **32 MiB**; তার বাইরে লিখলে পুরো ফাইলকে **সরাতে** হয়। আধুনিক সমাধান **এক্সটেন্ট (extent)** — (শুরু, দৈর্ঘ্য) রানের ছোট তালিকা।

৫ ইন্ডেক্সড অ্যালোকেশন ও UNIX ই-নোড

সংজ্ঞা: **ইন্ডেক্সড অ্যালোকেশন** প্রতিটি ফাইলকে একটি **ইন্ডেক্স ব্লক** দেয় — যার এন্ট্রিগুলো ফাইলের ডেটা ব্লকের ঠিকানা। লজিক্যাল ব্লক n-এ যেতে ইন্ডেক্স ব্লক পড়ে এন্ট্রি n নেয়: **বাইরের খণ্ডন নেই**, সরাসরি অ্যাক্সেস মেলে, ফাইল যেকোনো জায়গায় বাড়তে পারে। একমাত্র দুর্বলতা **ভেতরের খণ্ডন** — ৩ ব্লকের ফাইলেও পুরো একটি ইন্ডেক্স ব্লক নষ্ট। **বহু-স্তরীয় ইন্ডেক্সিং**-এ ইন্ডেক্স ব্লক আরেকটি ইন্ডেক্স ব্লকের দিকে তির করে, ফলে পৌঁচ n^k বাড়ে।



ই-নোডের ১৩টি ঠিকানা 512-B ব্লকে ≈ 1 GB ছোঁয়

স্তর	পৌঁচ	রিড
ডাইরেক্ট (১০)	ব্লক 0–9	1
সিঙ্গেল ইন্ডারেক্ট	+ 128 ব্লক	2
ডাবল ইন্ডারেক্ট	+ 16 384	3
ট্রিপল ইন্ডারেক্ট	+ 2 097 152	4
মোট	**2 113 674**	**≈ 1.01 GiB**

যেখানে **128** = ব্লক আকার ÷ পয়েন্টার আকার = 512 B ÷ 4 B। আধুনিক **ext2/ext4**-এ একই গঠন, কিন্তু **১২ ডাইরেক্ট + ১ + ১ + ১ = ১৫** পয়েন্টার; 4 KiB ব্লক ও 1 KiB ইন্ডারেক্ট ব্লকে (1 024 ঠিকানা) এটি 4 MiB + 4 GiB + 4 TiB ছোঁয়।

একটি **ই-নোড (i-node)** মেটাডেটা ও পয়েন্টার — দুটোই — রাখে। **হার্ড লিঙ্ক** হলো একই ই-নোডের দ্বিতীয় ডিরেক্টরি এন্ট্রি, তাই **nlink** 0 হলে তবেই ফাইল মরে; আর নাম ই-নোডে না থাকায় এক ফাইলের অনেক নাম থাকতে পারে।

সমাধানকৃত উদাহরণ ১ ✍ লজিক্যাল ব্লক ২০ খোঁজা: ডাইরেক্ট 0–9 ঢাকে, তাই ব্লক ২০ **সিঙ্গেল ইন্ডারেক্ট**: 20 − 10 = **10** → ওই ব্লকের **১১তম** ঠিকানা। খরচ: **২টি ডিস্ক রিড**।

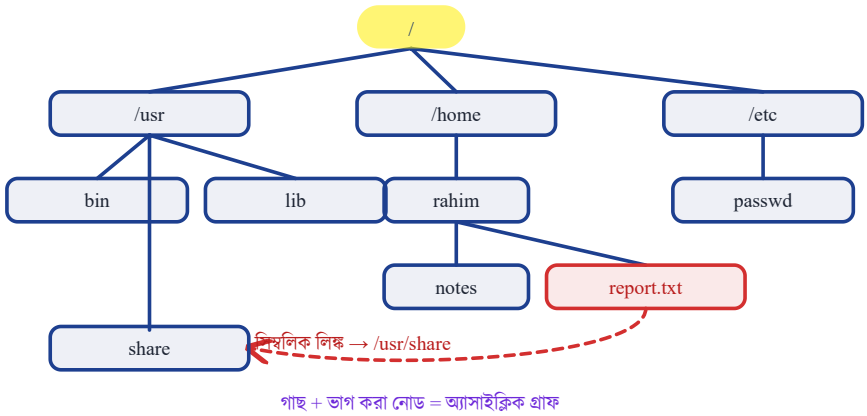
সমাধানকৃত উদাহরণ ২ ✍ লজিক্যাল ব্লক ৩০০ খোঁজা: ডাইরেক্ট + সিঙ্গেল মিলে মাত্র ১৩৮ ব্লক, তাই ৩০০ **ডাবল ইন্ডারেক্ট**: 300 − 138 = 162 → 162 ÷ 128 = 1 (২য় সাব-ব্লক), ভেতরে 162 mod 128 = **34**। খরচ: **৩টি রিড**।

মনে রাখার কৌশল: "স্তর বাড়াও, বিয়োগও করো।" প্রথম ১০ ডাইরেক্ট; 10 বিয়োগ করে সিঙ্গেল ইন্ডারেক্ট পড়ো; উত্তর এখনো ≥ 128 হলে আরেকবার 128 বিয়োগ করে ডাবল পড়ো।

পরীক্ষার ফাঁদ: ই-নোডে **নাম** থাকে? না — ডিরেক্টরিতে থাকে। **সুপারব্লক** ব্লক বিট ম্যাপ ও ই-নোড-টেবিলের পয়েন্টার রাখে, তাই সে ভলিউম-পর্যায়ের কাঠামো। **ফাইল ডেসক্রিপ্টর** একটি পূর্ণসংখ্যা সূচক, ই-নোড নয়।

৬ ডিরেক্টরির গঠন

সংজ্ঞা: **ডিরেক্টরি** এমন একটি বিশেষ ফাইল যার এন্ট্রি **নামের সঙ্গে শনাক্তকারীর** (ই-নোড নম্বর বা প্রথম ক্লাস্টার) সম্পর্ক রাখে। এর চারটি কাজ: **দল বাঁধা, নামকরণ, অনুসন্ধান, ভাগ**। কোনো ডিরেক্টরির নিজস্ব "লেখা" বিট ই চাইলে ভেতরে এন্ট্রি যোগ-বাদ দেওয়ার অনুমতি।



একটি ভাগ করা নোডসহ গাছ — UNIX-ধাঁচের অ্যাসাইক্লিক গ্রাফ

গঠন	আকৃতি	যা দেয়	দুর্বলতা
সিঙ্গেল-লেভেল	একটি সমতল তালিকা	সবচেয়ে সহজ	নামের সংঘর্ষ, দলবদ্ধতা নেই
টু-লেভেল	মাস্টার + প্রতি ব্যবহারকারীর	ব্যবহারকারীভিত্তিক বিচ্ছেদ	প্রকল্পভিত্তিক দল নেই
গাছ	অ্যাসাইক্লিক, যেকোনো গভীরতা	স্বাভাবিক স্তরবিন্যাস	ভাগে কপি বা লিঙ্ক লাগে
অ্যাসাইক্লিক গ্রাফ	গাছ + ভাগ করা ডিরেক্টরি	এক কপি, অনেক পিতা	ডেঙ্গে পড়তে পারে
জেনারেল গ্রাফ	চক্র অনুমোদিত	সর্বাধিক নমনীয়তা	ট্রাভার্সাল ঘুরতে পারে
হ্যাশ	h(নাম) → বাকেট	O(1) অনুসন্ধান	ক্রম নেই; বড় হলে খারাপ

পাথের নাম: নিরপেক্ষ */usr/bin/gcc*, আপেক্ষ *../docs/r.txt*।

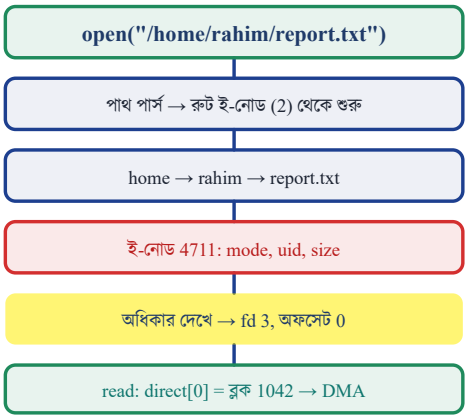
মনে রাখার কৌশল: সিঙ্গেল → টু → গাছ → অ্যাসাইক্লিক → জেনারেল → হ্যাশ।

পরীক্ষার ফাঁদ: সিম্বলিক লিঙ্ক **পাথ** রাখে, হার্ড লিঙ্ক **ই-নোড নম্বর**।

সমাধানকৃত উদাহরণ ✍️ একটি পাথের দাম কত?: */a/b/c/file* খোলা: রুটের ব্লক পড়ে **a** খুঁজি; a-র ব্লক পড়ে **b**; b-র ব্লক পড়ে **c**; c-র ব্লক পড়ে **file**; তারপর ই-নোড। **512-B** ব্লকে 16-বাইটের UNIX ডিরেক্টরি এন্ট্রি হলে এক ব্লকে **32টি এন্ট্রি**, তাই একটি ডেটা-বাইট নড়ার আগেই **৪টি ডিরেক্টরি রিড + ১টি ই-নোড রিড**।

৭ সমাধানকৃত ট্রেস: ফাইল খোলা

সংজ্ঞা: **open()** ফাইল পড়ে না — এটি **পাথ নামকে ই-নোডে** রূপান্তর করে, কলকারীর অধিকার যাচাই করে এবং একটি **ফাইল ডেসক্রিপ্টর** ফেরত দেয়। ডেটা ব্লক প্রথম read বা write-এ ছোঁয়।



একটি ডেটা-ব্লক ছোঁয়ার আগে সব উপাদান মিলে যায়

১. **পাথ পার্স**। এটি / দিয়ে শুরু, তাই হাঁটা শুরু **রুট** ডিরেক্টরি (ই-নোড 2) থেকে; আপেক্ষ পাথ হলে প্রক্রিয়ার **cwd** থেকে।

২. **উপাদানে হাঁটা**। রুটের ব্লক পড়ে **home** মেলে তার ই-নোড (30) নিই; এভাবে **rahim** (41) ও **report.txt** → ই-নোড **4711**।

৩. **ই-নোড আনো**। **সুপারব্লক**-এর ই-নোড-টেবিল পয়েন্টার ব্যবহার করে 4711 পড়ি: mode, uid, gid, nlink, size, পয়েন্টার।

৪. **সুরক্ষা যাচাই**। মোডের মালিক / গ্রুপ / অন্য বিট কলকারীর সঙ্গে মিলিয়ে দেখি; অনুমতি না থাকলে **EACCES**।

৫. **ওপেন-ফাইল এন্ট্রি বানাও**। ই-নোড পয়েন্টার, **O_RDONLY**, **অফসেট 0** ও রেফারেন্স কাউন্ট রাখি; **ডেসক্রিপ্টর** ফেরত দিই (**fd = 3**)।

৬. **read(fd, buf, 512)**। অফসেট 0 → লজিক্যাল ব্লক 0 → ই-নোডের প্রথম ডাইরেক্ট পয়েন্টার → **ব্লক 1042** → DMA করে অফসেট 512।

৭. **close(fd)**। বাফার ফাঁশ, ডেসক্রিপ্টর মুক্ত, রেফারেন্স কাউন্ট কমে; শূন্য হলে এন্ট্রি ও ই-নোড মুক্ত।

কল	ডিস্ক থেকে পড়ে	পরিবর্তন	ফেরত
open("/home/rahim/report.txt", O_RDONLY)	৩টি ডিরেক্টরি ব্লক + ই-নোড	একটি ওপেন-ফাইল এন্ট্রি, অফসেট 0	fd = 3
read(3, buf, 512)	ডেটা ব্লক 1042	অফসেট 0 → 512	512
lseek(3, 0, SEEK_END)	কিছু নয়	অফসেট ← size	8200
close(3)	কিছু নয়	ডেসক্রিপ্টর মুক্ত, রেফারেন্স −1	0

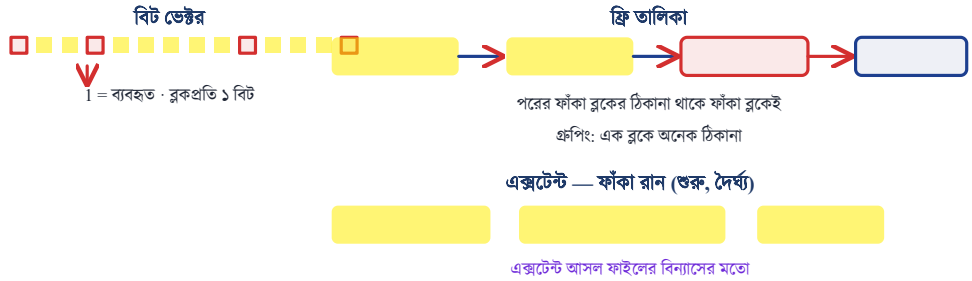
পরীক্ষার ফাঁদ: **open()** কোনো **ডেটা ব্লক** ছোঁয় না — শুধু ডিরেক্টরি ও ই-নোড। প্রতিটি উপাদান মেলা আবশ্যক; কোথাও নাম না মিললে পুরো কল **ব্যর্থ**, আংশিক খোলা হয় না।

মনে রাখার কৌশল: "মেলাও, নোট করো, ফেরত দাও — তারপর পড়ো": পাথ → ই-নোড → ওপেন-ফাইল টেবিল → ডেসক্রিপ্টর → ডেটা ব্লক।

IT link: শেল *./app --config conf.ini* চালালে ঠিক এই কাজটিই হয়: প্রতিটি উপাদান একটি ডিরেক্টরি হাঁটা, প্রতিটি open একটি ই-নোড লুকআপ, আর ডেসক্রিপ্টর প্রক্রিয়ার টেবিলের এন্ট্রি যা */proc/<pid>/fd*-তে দেখা যায়।

৮ ফ্রি-স্পেস ব্যবস্থাপনা

সংজ্ঞা: ফাইল সিস্টেমকে সবসময় বলতে হয় "পরের ফাঁকা ব্লকের রান কোথায়?"। **ফ্রি-স্পেস ব্যবস্থাপনা** সেই হিসাব; আর **সুরক্ষা (protection)** ঠিক করে কোন **বিষয় (ব্যবহারকারী, প্রক্রিয়া)** কোন **বস্তুর (ফাইল, ডিভাইস)** কী করতে পারবে।



"পরের ফাঁকা রান কোথায়?" — তিনটি উত্তর

পদ্ধতি	নোট
বিট ভেক্টর	দ্রুত, RAM-এ ক্যাশ হয়
লিঙ্কড তালিকা	বাড়তি জায়গা লাগে না
গ্রুপিং	এক ব্লকে অনেক ঠিকানা
এক্সটেন্ট	প্রতি রানে একটি এন্ট্রি

সমাধানকৃত উদাহরণ ✍️ বিট ভেক্টর কত বড়?: **1 GiB** ভলিউমে 512-B ব্লকে $2^{30} \div 2^{9} =$ **2 097 152** ব্লক, তাই ম্যাপ লাগে 2 097 152 ÷ 8 = **256 KiB** — ক্যাশে ধরে, এক পড়ায় লোড হয়। পয়েন্টার তালিকা 4 B করে চাইলে **8 MiB**, অর্থাৎ **32 গুণ** বড়।

৯ সুরক্ষা: ম্যাট্রিক্স, ACL ও ক্যাপাবিলিটি

সংজ্ঞা: **অ্যাক্সেস কন্ট্রোল ম্যাট্রিক্স**-এ সারি = বিষয়, কলাম = বস্তু; প্রতিটি ঘরে ওই জোড়ায় প্রোধিত অধিকার। শাস্ত্রীয় অধিকার: **পড়া, লেখা, চালানো, যোগ করা, মুছে ফেলা, মালিক বদলানো**। এটি কখনো ঘন হিসেবে রাখা যায় না, তাই দুই ধরনের পাতলা রূপ ব্যবহার হয়।

রূপ	কোথায় রাখা	সমস্যা
অ্যাক্সেস তালিকা (ACL)	প্রতি *ফাইলে*: (ব্যবহারকারী বা গ্রুপ, অধিকার)	প্রতি ফাইলে প্রতি ব্যবহারকারী
ক্যাপাবিলিটি তালিকা	প্রতি *প্রক্রিয়ায়*: জাল করা যায় না এমন টোকেন	হাতে হাতে বাতিল করতে হয়
UNIX মোড	ই-নোডে ৯ বিট: মালিক / গ্রুপ / অন্য	নাম ধরা ব্যবহারকারী নেই; rwx = 4 + 2 + 1
NTFS ACE	MFT রেকর্ডে: প্রতি SID-এর allow / deny	আগে deny মূল্যায়ন হয়

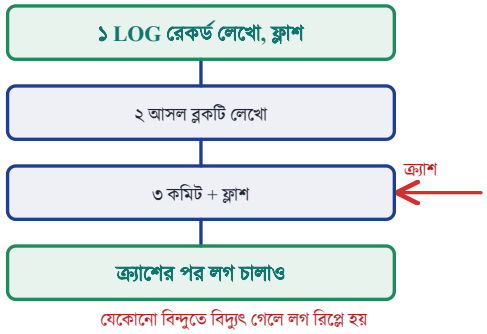
সমাধানকৃত উদাহরণ ✍️ মোড: *chmod 640 notes.txt* → মালিক **rw-** (6), গ্রুপ **r--** (4), অন্য **---** (0)। বিশেষ বিট: setuid **4000**, setgid **2000**, sticky **1000** — তাই */tmp* হয় 1777।

পরীক্ষার ফাঁদ: **ম্যাট্রিক্সই মডেল**; ACL প্রতি **বস্তুতে**, ক্যাপাবিলিটি প্রতি **বিষয়ে**। সুরক্ষা মানে **অধিকার** — পরিচয় ও এনক্রিপশন হলো **নিরাপত্তা**।

১০ ফাইল-সিস্টেমের প্রকার ও জার্নালিং

বিষয়	FAT12	FAT16	FAT32
প্রতি এন্ট্রির বিট	**12**	**16**	**32** (28 = ক্লাস্টার নং)
সর্বোচ্চ ক্লাস্টার	2^{12} = **4 096**	2^{16} = **65 524**	$2^{28} \approx$ **268 M**
ক্লাস্টার আকার	512 B	512 B–32 KB	512 B–32 KB
সর্বোচ্চ ভলিউম	≈ **2 MB**	≈ **32 MB** (2 GB)	128 GiB … **2 TB**
নাম	8.3	8.3	8.3 + লম্বা নাম (VFAT)
কোথায় ব্যবহার	1.44 MB ফ্লপি	১৯৮০-৯০ দশকের ডিস্ক	Windows 95 OSR2 থেকে

সিস্টেম	জার্নাল	বৈশিষ্ট্য
FAT12 / FAT16	নেই	লিঙ্কের টেবিল, 8.3 নাম
ext2 (1992)	নেই	15-পয়েন্টার ই-নোড, ব্লক গ্রুপ
ext3 (2001)	**আছে**	ext2 + মেটাডেটা জার্নাল
ext4 (2006)	আছে	এক্সটেন্ট, 64-বিট, চেকসাম
NTFS (1993)	আছে	MFT, ACL, স্ট্রিম, EFS
XFS	মেটাডেটা	64-বিট, অলোকেশন গ্রুপ



write-ahead logging: পরিবর্তনের আগেই উদ্দেশ্য স্থায়ী

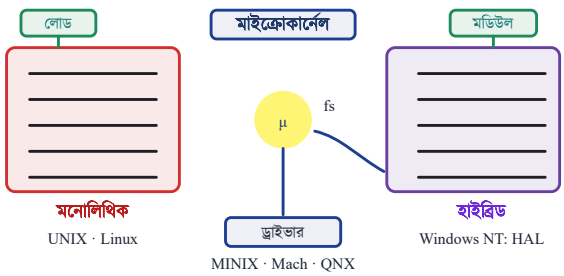
জার্নালিং আসলে কী?: প্রস্তাবিত **মেটাডেটা** পরিবর্তনের একটি **write-ahead log**, পরিবর্তনের **আগেই** স্থায়ী স্টোরেজে ফ্লাশ করা; পরের মাউন্টে সেটি **রিপ্লে** হয়ে বাদ দেওয়া হয়। তাই ক্র্যাশে ফাইল সিস্টেম **সামঞ্জস্যপূর্ণ** থাকে, ডেটা অক্ষত থাকে না।

জার্নাল ছাড়া পুনরুদ্ধার: বুটের সময় **fsck** (Linux-এ **e2fsck**) ও **chkdsk** চলে: সুপারব্লক, বিট ম্যাপ, ই-নোড-টেবিল ও ফ্রি-ব্লক গণনা যাচাই করে, ফ্রি-স্পেস ম্যাপ গড়ে। এটি O(ভলিউমের আকার) — জার্নাল যে খরচই বাঁচায়।

মনে রাখার কৌশল: "ext**3** = ext**2** + একটি **লগ** (2001); ext**4** = এক্সটেন্ট + 64-বিট + চেকসাম (2006)।" **NTFS 1993**; **XFS** = অলোকেশন গ্রুপ।

পরীক্ষার ফাঁদ: FAT12-এর সীমা **4 096 ক্লাস্টার** (≈ 2 MB); 65 524 FAT16-এর। জার্নাল **মেটাডেটা** রক্ষা করে, ডেটা নয়।

১১ অপারেটিং সিস্টেমের কেস স্টাডি



তিন ধরনের কার্নেল; Linux মনোলিথিক, তবু লোডযোগ্য

MS-DOS (1981): এক-ব্যবহারকারী, **এক-টাস্কিং**, 8086-এর রিয়েল মোড, **640 KB** মেমরি, **কোনো মেমরি সুরক্ষা নেই**। FAT12 / FAT16, 8.3 নাম, শেল **COMMAND.COM**।

UNIX (Bell Labs, 1969–70): কেন থম্পসন ও ডেনিস রিচি; সংস্করণ ১ **1970**-এ, 1972-এ নাম "Unix", 1973-এ **C**-তে লেখা। বহু-ব্যবহারকারী, বহু-টাস্কিং; **সবকিছুই একটি ফাইল**; **পাইপ (pipe)** কমান্ডের ধাপ জোড়া লাগায়; ই-নোড ফাইল সিস্টেম।

Windows NT (1993): একটি **হাইব্রিড কার্নেল** — মাইক্রোকার্নেল কোর, বড় এক্সিকিউটিভ, ইউজার-মোড সাবসিস্টেম — সঙ্গে হার্ডওয়্যার লুকানোর **HAL**। এর চিহ্ন **রেজিস্ট্রি (registry)**, যা DOS-এর টেক্সট ফাইলের জায়গা নিল; NTFS-এ **SID**-ভিত্তিক ACL।

Linux (ঘোষিত ১৪ আগস্ট ১৯৯১): লিনাস টরভাল্ডস, অনুপ্রেরণা MINIX ও UNIX থেকে। একটি **মনোলিথিক কার্নেল**, কিন্তু চলার সময় **লোডেবল কার্নেল মডিউলের** সাহায্যে মডিউলার (**insmod/rmmod**)। **সবকিছুই একটি ফাইল**; **VFS** ext4 ও XFS-কে একরকম দেখায়।

বিষয়	MS-DOS	UNIX	Windows NT	Linux
উৎপত্তি	IBM / MS, 1981	Bell Labs, 1969	Microsoft, 1993	Torvalds, 1991
কার্নেল	সুরক্ষিত নয়	মনোলিথিক	**হাইব্রিড**	মনো + **মডিউল**
ব্যবহারকারী ও টাস্ক	1 / 1	অনেক / অনেক	অনেক / অনেক	অনেক / অনেক
কনফিগারেশন	CONFIG.SYS	/etc-র টেক্সট	**রেজিস্ট্রি**	/etc-র টেক্সট
ফাইল সিস্টেম	FAT12 / FAT16	ই-নোড, ext4	**NTFS**, FAT	ext4, XFS
সবকিছুই ফাইল?	না	**হ্যাঁ**	না	**হ্যাঁ**
সুরক্ষা	কিছু নয়	rwx, গ্রুপ	ACL, SID	rwx, গ্রুপ

পরীক্ষার ফাঁদ: **Windows 1.0 (1985)** ছিল **DOS-এর ওপর একটি GUI শেল**, বহু-টাস্কিং সিস্টেম নয় — সেই ধারা **Windows NT (1993)**। আর **রেজিস্ট্রি ফাইল সিস্টেম নয়**।

মনে রাখার কৌশল: "UNIX ও Linux-এ সবকিছু **ফাইল**, Windows একটি **ডেটাবেস**।" কার্নেল: মনোলিথিক · মাইক্রোকার্নেল · হাইব্রিড।

পরীক্ষার আগের রাতের পাতা

তথ্য-পাতা:

ব্লক আকার 512 B – 4 KiB

৮ গুণাবলি, ৭ অপারেশন

কনটিগিউয়াস: base + n, বাইরের খণ্ডন

লিঙ্কড: শৃঙ্খল বেয়ে যাওয়া

FAT = লিঙ্কড, লিঙ্ক টেবিলে

FAT 0 = ফাঁকা, 0xFFF = শেষ

FAT12 2^{12} = 4 096 ক্লাস্টার ≈ 2 MB

FAT16 2^{16} = 65 524 ≈ 32 MB

FAT32 28-বিট ক্লাস্টার নং ≈ 2 TB

ফ্লপি: 2 880 সেক্টর → 2 847 ডেটা

ই-নোড: 10 ডাইরেক্ট + 1, 1, 1

512-B ইন্ডারেক্টে 128 ঠিকানা

10+128+16 384+2 097 152 ব্লক

= 2 113 674 ≈ 1.01 GiB

ext2/4: 12 ডাইরেক্ট + 1 + 1 + 1

UNIX এন্ট্রি 16 B, ব্লকে 32টি

সিম্বলিক = পাথ, হার্ড = ই-নোড

1 GiB/512-B: 2 097 152 → 256 KiB

ফ্রি: বিট ভেক্টর, তালিকা, এক্সটেন্ট

ACL প্রতি বস্তুতে, ক্যাপাবিলিটি বিষয়ে

rwx = 4+2+1; 4000/2000/1000

ext2 1992 · ext3 2001 · ext4 2006

NTFS 1993: MFT, ACL, $LogFile

জার্নাল = write-ahead log, মেটাডেটা

fsck / e2fsck / chkdsk

DOS 1981 · UNIX 1969 · NT 1993

Linux 1991; NT কার্নেল = হাইব্রিড

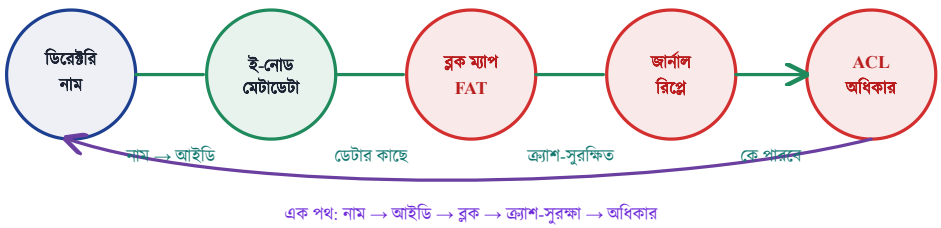
UNIX ও Linux: সবকিছুই ফাইল

MCQ-র জন্য সবচেয়ে জরুরি ১২টি বাক্য

- ফাইলের **নাম** ডিরেক্টরিতে, ই-নোডে নয়
- প্রমিত অপারেশন = **৭**
- কনটিগিউয়াস এলোমেলো = base + n
- কনটিগিউয়াসে **বাইরের** খণ্ডন
- FAT = লিঙ্কড, লিঙ্ক টেবিলে
- FAT12 সর্বোচ্চ = **4 096** ক্লাস্টার
- ext3 = ext2 + জার্নাল
- জার্নাল **মেটাডেটা** রক্ষা করে, ডেটা নয়
- ACL প্রতি বস্তুতে, ক্যাপাবিলিটি বিষয়ে
- হার্ড লিঙ্ক = ই-নোড নং, সিম্বলিক = পাথ
- NT-তে DOS-এর ফাইলের জায়গায় **রেজিস্ট্রি**
- UNIX ও Linux: **সবকিছুই ফাইল**

সবচেয়ে বড় ফাঁদ: FAT12 = 4 096 ক্লাস্টার, 65 524 নয় · FAT কনটিগিউয়াস বা ইন্ডেক্সড নয় · ই-নোডে ফাইলের নাম নেই · 512 ÷ 4 = 128 ঠিকানা · বিট ভেক্টরে n/8 বাইট · জার্নাল মেটাডেটা রক্ষা করে · Windows 1.0 (1985) ছিল DOS-এর GUI।

সব কৌশল একসঙ্গে: ই-নোডে নাম ছাড়া সব · FAT = সরানো লিঙ্ক · কনটিগিউয়াস = base + n, তাই দ্রুত · স্তর বাড়াও, বিয়োগও করো · সিম্বলিক পাথ রাখে, হার্ড লিঙ্ক ই-নোড নং · ext3 = ext2 + লগ · UNIX ফাইল, Windows ডেটাবেস।



এক পথ: নাম → আইডি → ব্লক → ক্র্যাশ-সুরক্ষা → অধিকার

পাঁচটি শব্দে পুরো ফাইল সিস্টেম

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. প্রমিত ফাইল অপারেশন কতটি?

ক) ৫ খ) ৭
গ) ৯ ঘ) ৪

২. UNIX/Linux-এ ফাইলের মেটাডেটা ও ব্লক পয়েন্টার থাকে কোথায়?

ক) সুপারব্লকে খ) বুট ব্লকে
গ) ই-নোডে ঘ) ফাইল ডেসক্রিপ্টরে

৩. FAT কোন অ্যালোকেশন পদ্ধতির রূপ?

ক) লিঙ্কড খ) কনটিগিউয়াস
গ) ইন্ডেক্সড ঘ) বহু-স্তরীয় ইন্ডেক্সড

৪. 512-B ব্লকে 16-বাইটের UNIX ডিরেক্টরি এন্ট্রি ধরলে কতটি ধরে?

ক) ১৬ খ) ৬৪
গ) ৮ ঘ) ৩২

৫. ext3, ext2 থেকে কী যোগ করে?

ক) ই-নোড টেবিল খ) জার্নাল (লগ)
গ) ক্লাস্টার ঘ) FAT

৬. FAT12 সর্বোচ্চ কতটি ক্লাস্টার ঠিকানা করতে পারে?

ক) ৪ ০৯৬ খ) ৬৫ ৫২৪
গ) 2^{28} ঘ) 2^{16}

৭. (শুরু, দৈর্ঘ্য) জোড়ায় ফাঁকা জায়গা রাখাকে কী বলে?

ক) বিট ভেক্টর খ) লিঙ্কড তালিকা
গ) এক্সটেন্ট ঘ) ই-নোড

৮. 512-B ব্লকের 1 GiB ভলিউমে বিট ভেক্টর কত?

ক) 64 KiB খ) 128 KiB
গ) 512 KiB ঘ) 256 KiB

৯. Windows NT, DOS-এর টেক্সট কনফিগারেশন ফাইলের জায়গায় কী ব্যবহার করে?

ক) CONFIG.SYS খ) রেজিস্ট্রি
গ) MFT ঘ) ব্যাচ ফাইল

১০. কনটিগিউয়াস ফাইল ব্লক ১২ থেকে শুরু; ৫ম ব্লক কোনটি?

ক) ব্লক ৫ খ) ব্লক ১২
গ) ব্লক ১৬ ঘ) ব্লক ১৭

উত্তরমালা → ১-খ · ২-গ · ৩-ক · ৪-ঘ · ৫-খ · ৬-ক · ৭-গ · ৮-ঘ · ৯-খ · ১০-গ

লিখিত প্রশ্ন: একটি UNIX ফাইল সিস্টেমে ব্লক 512 বাইট, পয়েন্টার 4 বাইট এবং প্রথাগত ই-নোড আছে — 10টি ডাইরেক্ট ঠিকানা এবং একটি করে সিঙ্গেল, ডাবল ও ট্রিপল ইন্ডারেক্ট। (ক) সবচেয়ে বড় ফাইল কত? (খ) লজিক্যাল ব্লক ২০ ও ৩০০ কোথায়? (গ) FAT-এর দুই সুবিধা ও দুই অসুবিধা। ১০

১. **প্রতি ইন্ডারেক্ট ব্লকে** — 512 ÷ 4 = **128** ঠিকানা। 1
২. **সর্বোচ্চ ফাইল** — 10 + 128 + 16 384 + 2 097 152 = **2 113 674 ব্লক ≈ 1.01 GiB**। 2
৩. **ব্লক ২০** — ডাইরেক্ট 0–9 ঢাকে, তাই **সিঙ্গেল ইন্ডারেক্ট**, সূচক 20 − 10 = 10; **২টি রিড**। 2
৪. **ব্লক ৩০০** — ডাইরেক্ট + সিঙ্গেল মিলে ১৩৮, তাই **ডাবল ইন্ডারেক্ট**, 300 − 138 = 162, সাব-ব্লক 1, ঠিকানা 34; **৩টি রিড**। 2
৫. **FAT — সুবিধা** ডেটা না পড়ে শৃঙ্খল বেয়ে যাওয়া যায়; যেকোনো এলোমেলো অ্যাক্সেসে একটি লুকআপই যথেষ্ট। **অসুবিধা** টেবিল প্রতি ভলিউমে, তাই ডিস্ক বড় হলে বাড়ে। 3

পরীক্ষার আগে যাচাই ✓

- ৮ গুণাবলি · ৭ অপারেশন
- কনটিগিউয়াস / লিঙ্কড / ইন্ডেক্সড
- ই-নোডের হিসাব · open থেকে close
- FAT সীমা · জার্নাল · চারটি OS